

FL-04 — Agents, MCP, and What My Pipeline Is Missing

Parth Chawla · B.Tech CSE (AI & DS) · 22 Sept 2026 · Track: Write backend code / reliable AI systems

1. What an agent actually is (in my own words)

A chatbot answers one prompt with one response. An **agent** closes a loop: it takes a goal, looks at the current state of the world, picks a tool to act with, reads the result, and decides what to do next — repeating until the goal is done or it gets stuck and asks for help. The loop is *perceive* → *decide* → *act* → *observe*, and the key property is that the **model chooses the next step at runtime** instead of following a path a programmer fixed in advance.

Three things have to be present before I call something an agent. First, **autonomy over steps**: given "clear my unread backlog," it decides whether to list mail, read a message, label it, or skip it — I did not hardcode that order. Second, **tool use with feedback**: it calls real functions (APIs, file writes, queries) and the observation from each call changes its next decision. Third, **a termination rule**: it knows when to stop — goal reached, budget spent, confidence too low — and says so instead of looping forever. A system with all three earns the name. A system with a fixed sequence of LLM calls, however clever each call is, does not — that is a workflow wearing an agent costume, and the distinction matters because the failure modes are completely different. Workflows fail when a step's input drifts; agents fail when the loop itself misjudges, repeats, or burns budget.

2. Workflow vs agent — and what my FL-04 pipeline is

A **workflow** is a path I draw in advance: step A always runs, then step B, with fixed branches I wrote. The model may do smart work *inside* a step, but it never reorders the steps. An **agent** gets the goal and the toolbox and draws its own path per run, including paths I never imagined.

My FL-04 pipeline — the Books to Scrape scraper in this repo (backend/Assignment 5: catalogue pages → fetch → parse with BeautifulSoup → validate with Pydantic → repair once → quarantine failures → write books.json + run-report.json) — is a **workflow, clearly**. Every run executes the same stages in the same order; retries happen exactly once and only for timeout/5xx; a 404 is never retried by design. There is no step where the model surveys state and picks a different strategy. That rigidity is a feature: it gives me idempotent output (60 records every run), a 0.5s politeness guarantee, and a failure-isolation promise (one broken page never kills the run). An agent would be free to, say, re-crawl everything because one price looked odd — my pipeline cannot, and my evaluation depends on it not doing so.

3. What MCP is, and its three primitives

The Model Context Protocol is an open standard for plugging external capabilities into an LLM app without custom glue per integration — often described as "USB-C for AI context." The app I chat in is the **host**; the connector inside it is the **client**; each external capability is a **server** speaking JSON-RPC with capability discovery. I read the current spec (2026-07-28) to write this, and the core idea is small: servers expose exactly three primitives. **Tools** are functions the model can call (`tools/call`) — they *do* things: query a DB, hit an API, write a file. The model decides when to call them, and calls can need my approval. **Resources** are read-only data the app feeds the model as context (`resources/read`) — file contents, schemas, records — addressed by URI, with templates for parameterized reads. The app, not the model, controls which resources are attached. **Prompts** are reusable instruction templates (`prompts/get`) that bundle a task pattern with the right tools and resources, invoked explicitly by me. Concretely: a database server can expose the schema as a resource, querying as a tool, and a "weekly-report" analysis pattern as a prompt — context, action, and procedure kept separate so each side stays honest about who chose what.

4. One concrete upgrade that would make FL-04 an agent

My pipeline needs exactly one architectural change to cross the line: **replace the fixed stage order with a model-driven control loop over the same toolbox**. Today the orchestrator calls fetch, then parse, then validate. In the agent version, the model would see the goal ("60 valid records, no extra requests, politeness respected") plus live state (cache hits, failures so far, run-report deltas) and choose each next tool call itself — e.g., noticing three detail pages failed with the same selector, hypothesizing a DOM change, and choosing to re-inspect one page's HTML before rewriting the selector, rather than following my hardcoded "repair once then quarantine" rule. Everything else — the tools, the validators, the politeness budget — stays identical; only *who picks the next step* changes from me-at-build-time to the model-at-runtime, fenced by a step budget (say, 40 tool calls) and a stop rule (all 60 valid, or report shortfall and quit). That single change converts deterministic

replayability into adaptive recovery — the honest tradeoff my proof statement ("reliable at scale") would then have to evaluate.

5. Evidence: one working connector, three tasks chat alone could not do

*Primary evidence — archived in this submission: evidence/Communication.pptx (12 slides: "Communication — Theory, Types & Workplace Practice") plus the full tool-call transcript evidence/session-ses_f647.md (74 tool calls, 13 Sept 2026). Honest correction to an earlier draft of this section: the deck was **not** built through the 2slides MCP — the transcript shows it was built through the agent harness's code-execution connector (bash tool running Python). That still satisfies the brief ("any MCP client counts"): the pattern is identical — the model discovers nothing by chatting, calls a tool, reads machine output, and acts on it. Each step below is a real tool call whose output chat alone could never produce.*

Task 1 — generate a binary artifact (tool: bash → scripts/generate_ppt.py). Chat alone cannot emit a valid .pptx binary. The call `uv run scripts/generate_ppt.py --spec .tmp/yukti_spec.json` returned SUCCESSFULLY GENERATED PRESENTATION ... 12 slides rendered — a real file on disk that I unzipped and verified (12 slide XML parts).

Task 2 — read back binary state (tools: bash → python-pptx / PIL / cv2). Chat alone cannot inspect pixels or XML. Follow-up calls opened the deck programmatically: transition flags per slide, image parts (/ppt/media/image1.jpg 338KB), background contour analysis (bg2.jpg minimalist vs bg1.jpg complex edges), resizing backgrounds to 1920×1080 — machine observations that drove the next edits.

Task 3 — diagnose the live environment (tool: bash → lockfile probe). Chat alone cannot know what is open on my machine. When saves failed with `PermissionError`, the loop probed `.tmp\yukti\.\~lock.Communication.pptx#` and concluded the file was locked open in ONLYOFFICE — then routed output to `Communication_landscape.pptx` instead. Acting on environment feedback, not text prediction.

Supplementary grounding for section 3 (same pattern — tool output, not chat): live fetch of books.toscrape.com page-1 and live read of the MCP spec (2026-07-28) with its tools/list + tools/call discovery-then-execution flow.

Word count (sections 1–4): ~850. Screenshots: the transcript's Tool: bash blocks (Input JSON + Output) are the tool-use proof; attach 2–3 of them with the card alongside the .pptx.